

# Plan de Implementación: Consultas SQL Centralizadas

## Objetivo

Centralizar las consultas SQL importantes en clases dedicadas para:

1. **Reutilización:** Los endpoints y la IA usan las mismas consultas
2. **Mantenibilidad:** Un solo lugar para modificar cada consulta
3. **Documentación:** Anotaciones `@ai-*` describen cada consulta para la IA

---

## Cambios Propuestos

### Fase 1: Crear estructura base (SIN modificar endpoints)

[NEW] `app/queries/BaseQuery.php` Clase base con métodos helper para todas las queries.

[NEW] `app/queries/OrdenesQueries.php` Consultas relacionadas con órdenes.

[NEW] `app/queries/ProduccionQueries.php` Consultas de lotes, tareas, empleados en producción.

[NEW] `app/queries/PagosQueries.php` Consultas de pagos, comisiones, abonos.

[NEW] `app/queries/InventarioQueries.php` Consultas de inventario y movimientos.

---

### Fase 2: Migrar endpoints gradualmente

[!CAUTION] Esta fase se hace endpoint por endpoint, probando cada cambio antes de continuar.

1. Identificar una consulta en un endpoint
  2. Moverla a la clase correspondiente
  3. Actualizar el endpoint para usar la clase
  4. **Probar el endpoint**
  5. Commit si funciona
- 

## Estructura de las Clases Query

```
<?php
namespace App\Queries;

/**
 * Consultas relacionadas con pagos y comisiones
 */
class PagosQueries
{
    /**
```

```

    * Calcular comisiones pendientes de un empleado
    *
    * @ai-description Calcula el total de comisiones pendientes de pago para un empleado
    * @ai-keywords comision, pago, empleado, pendiente, deuda
    * @ai-ejemplo ¿Cuánto le debo a María? | Comisiones pendientes de Juan
    *
    * @param int $idEmpleado ID del empleado
    * @param string|null $fechaInicio Fecha inicio del período (opcional)
    * @param string|null $fechaFin Fecha fin del período (opcional)
    * @return string SQL query
    */
    public static function comisionesPendientesEmpleado(
        int $idEmpleado,
        ?string $fechaInicio = null,
        ?string $fechaFin = null
    ): string {
        $sql = "
            SELECT
                SUM(ld.comision * op.precio_unitario * ld.unidades_solicitadas) as total_comis
            FROM lotes_detalle ld
            INNER JOIN ordenes_productos op ON ld.id_ordenes_productos = op._id
            INNER JOIN ordenes o ON ld.id_orden = o._id
            WHERE ld.id_empleado = {$idEmpleado}
            AND ld.terminado = 1
            AND (ld.pago_comision IS NULL OR ld.pago_comision = 'pendiente')
        ";

        if ($fechaInicio && $fechaFin) {
            $sql .= " AND DATE(ld.fecha_terminado) BETWEEN '{$fechaInicio}' AND '{$fechaFin}'"
        }

        return $sql;
    }
}

```

---

## Cómo la IA Usará las Consultas

### Script de extracción automática

Un script PHP leerá las anotaciones @ai-\* de las clases y generará el contexto:

```

// app/schemas/gemini/QueryExtractor.php
class QueryExtractor {
    public static function extractFromClass(string $className): array {
        $reflection = new ReflectionClass($className);
        $queries = [];
    }
}

```

```

        foreach ($reflection->getMethods() as $method) {
            $doc = $method->getDocComment();
            if (strpos($doc, '@ai-description') !== false) {
                $queries[] = [
                    'nombre' => $method->getName(),
                    'descripcion' => self::extractTag($doc, '@ai-description'),
                    'keywords' => self::extractTag($doc, '@ai-keywords'),
                    'ejemplos' => self::extractTag($doc, '@ai-ejemplo'),
                ];
            }
        }

        return $queries;
    }
}

```

---

## Orden de Implementación

### Paso 1: Crear clases vacías

- ☐ app/queries/BaseQuery.php
- ☐ app/queries/OrdenesQueries.php
- ☐ app/queries/ProduccionQueries.php
- ☐ app/queries/PagosQueries.php
- ☐ app/queries/InventarioQueries.php

### Paso 2: Identificar consultas críticas

- ☐ Revisar app/routes/payments.php → consultas de pagos
- ☐ Revisar app/routes/reports.php → consultas de reportes
- ☐ Revisar app/routes/manufacturing.php → consultas de producción

### Paso 3: Migrar consultas una por una

- ☐ Migrar primera consulta de pagos → probar
- ☐ Migrar primera consulta de producción → probar
- ☐ Continuar gradualmente

### Paso 4: Integrar con IA

- ☐ Crear extractor de anotaciones
  - ☐ Modificar GeminiAssistant para usar ejemplos de queries
- 

## Verificación

[!IMPORTANT] Después de cada migración de consulta, verificar: 1. El endpoint sigue funcionando igual 2. Los datos retornados son idénticos 3. No hay errores en logs

---

## Rollback

Si algo sale mal:

```
git checkout -- app/routes/archivo_afectado.php
```

O revertir todo el commit si es necesario.